COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/30/2026	**DATE SUBMITTED:**	9/30/2026
NAME:	Kherwin Emmanuel D. Milan	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Screenshot	
Observations	The program makes an array of 100 numbers with random values from 0 to 99. The numbers are not in order. displayArr prints all 100 numbers. I copied this array for each sort, so all three sorting algorithms start with the same values.

```
const int maxSize = 100;

void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

int main() {
    int unsortedArr[maxSize];
    srand(time(0));

    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand() % 100;
    }
    std::cout << "Original Unsorted Array (100 Elements):\n";
    displayArr(unsortedArr);

    int shellArr[maxSize], mergeArr[maxSize], quickArr[maxSize];
    for (int i = 0; i < maxSize; i++) {
        shellArr[i] = unsortedArr[i];
        mergeArr[i] = unsortedArr[i];
        quickArr[i] = unsortedArr[i];
    }
```

```
Original Unsorted Array (100 Elements):
73 70 93 29 40 0 85 23 53 99 79 57 85 22 80 17 97 41 38 37 56 56 19 83 57 89 73 65 81 10 59
 11 20 6 7 7 97 12 45 38 15 8 69 12 28 35 32 20 73 41 29 85 44 9 85 35 6 83 15 3 21 4 86 72
 3 51 79 99 31 35 3 94 47 62 89 10 91 8 52 46 41 31 32 22 82 1 89 21 72 18 32 35 21 83 29 5
8 66 59 4 71
```

Table 8-1. Array of Values for Sort Algorithm Testing

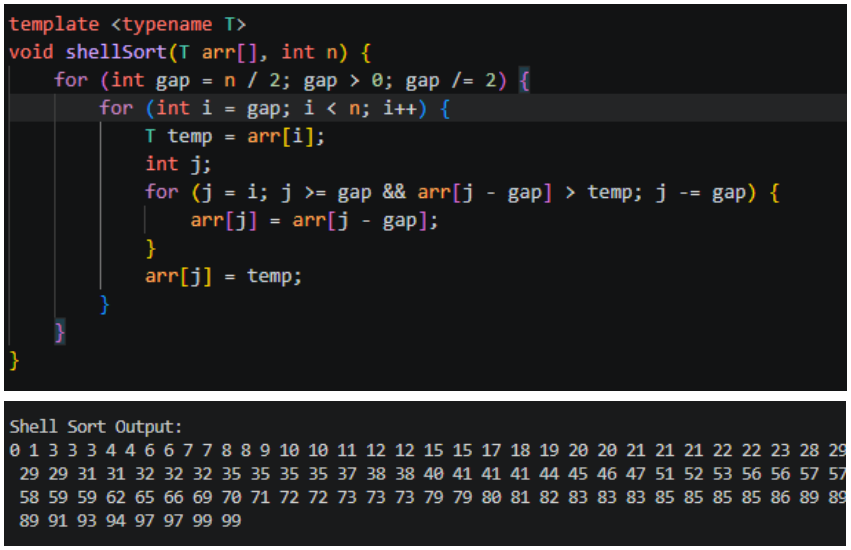
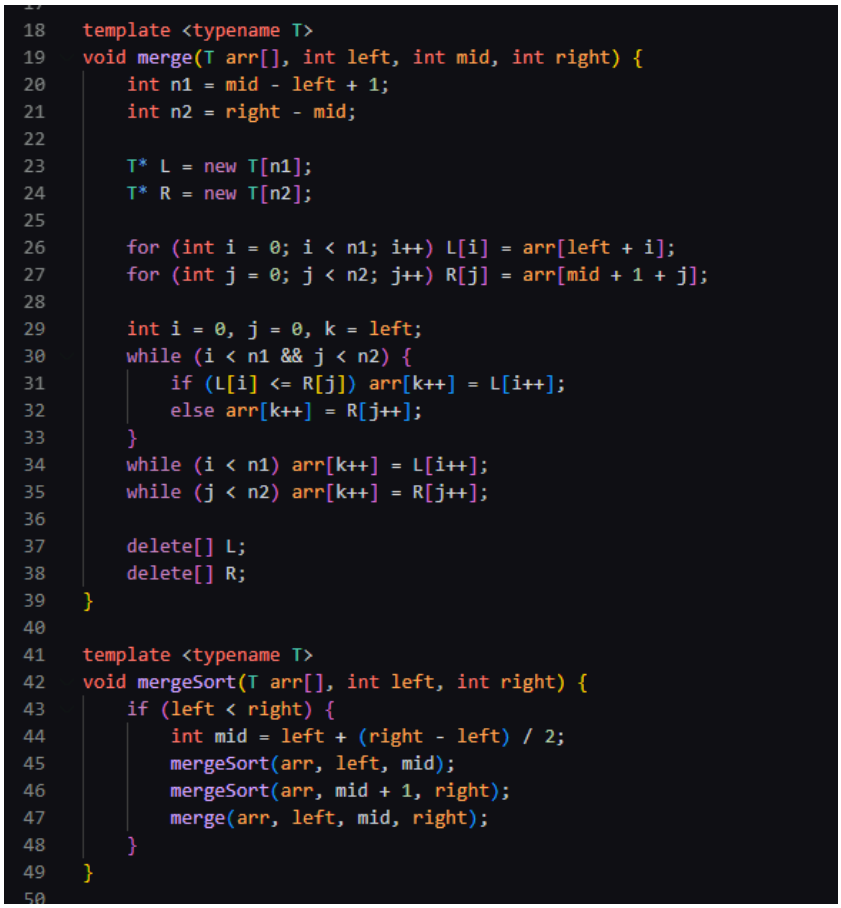
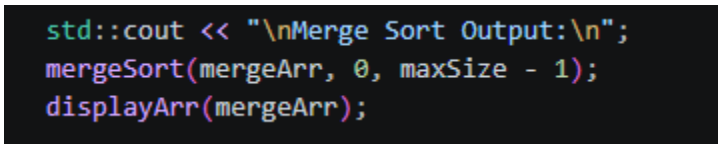
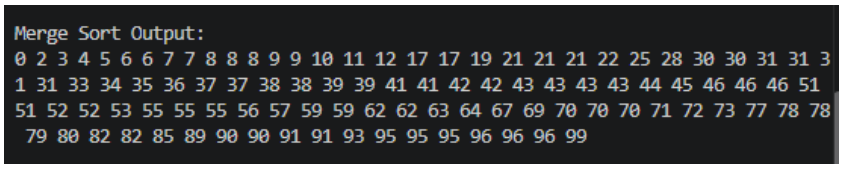
Code + Console Screenshot	
Observations	Shell sort starts with a gap of 50 (n/2). The gap is cut in half each round until it is 1. In each round it does insertion sort on the numbers that are a gap apart. The output is sorted from smallest to biggest. It is like insertion sort, but the numbers move farther in one step.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	  
Observations	Merge sort splits the array in the middle again and again until each part has one number. Then the merge function puts the parts back together in order, using the temporary arrays L and R. The output is sorted and it is the same as the shell sort output.

```
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    delete[] L;
    delete[] R;
}

template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
std::cout << "\nMerge Sort Output:\n";
mergeSort(mergeArr, 0, maxSize - 1);
displayArr(mergeArr);
```

```
Merge Sort Output:
0 2 3 4 5 6 6 7 7 8 8 8 9 9 10 11 12 17 17 19 21 21 21 22 25 28 30 30 31 31 3
1 31 33 34 35 36 37 37 38 38 39 39 41 41 42 42 43 43 43 43 44 45 46 46 46 51
51 52 52 53 55 55 55 56 57 59 59 62 62 63 64 67 69 70 70 70 71 72 73 77 78 78
 79 80 82 82 85 89 90 90 91 91 93 95 95 95 96 96 96 99
```

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	
Observations	Quick sort uses the last number as the pivot. The partition function moves smaller numbers to the left of the pivot and bigger numbers to the right. Then quickSort calls itself on the left side and the right side. The output is sorted and it is the same as the other two sorts, so I think my code works

```
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high]; // Last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

```
std::cout << "\nQuick Sort Output:\n";
quickSort(quickArr, 0, maxSize - 1);
displayArr(quickArr);
```

```
Quick Sort Output:
0 2 3 4 5 6 6 7 7 8 8 8 9 9 10 11 12 17 17 19 21 21 21 22 25 28 30 30 31 31 3
1 31 33 34 35 36 37 37 38 38 39 39 41 41 42 42 43 43 43 43 44 45 46 46 46 51
51 52 52 53 55 55 55 56 57 59 59 62 62 63 64 67 69 70 70 70 71 72 73 77 78 78
 79 80 82 82 85 89 90 90 91 91 93 95 95 95 96 96 96 99
```

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes. After partition, the left side and the right side are separate lists, so we can sort them with any sorting algorithm, like insertion sort or selection sort. The pivot is already in its correct place.

Example: {24, 9, 29, 14, 45, 18}
Step 1: Partition using pivot 18. Left is {9, 14}, pivot is 18, right is {24, 45, 29}.
Step 2: Sort the left side with insertion sort, which gives {9, 14}. Sort the right side with selection sort, which gives {24, 29, 45}.
Step 3: Put them together (left, pivot, right): {9, 14, 18, 24, 29, 45}.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- Quick sort, I think, out of the three sorts in this activity. It sorts in place, so it does not need extra arrays like merge sort does. The array only has 15 elements, so the time difference is very small. I did not measure it.

Conclusion:
In this activity I learned how shell sort, merge sort and quick sort work. Shell sort uses gaps, merge sort splits and merges, and quick sort uses a pivot. I tested all of them on a random array of 100 numbers, and the outputs came out sorted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.